



MODEL
CONTEXT
PROTOCOL



LEARN



BUILD



INTEGRATE

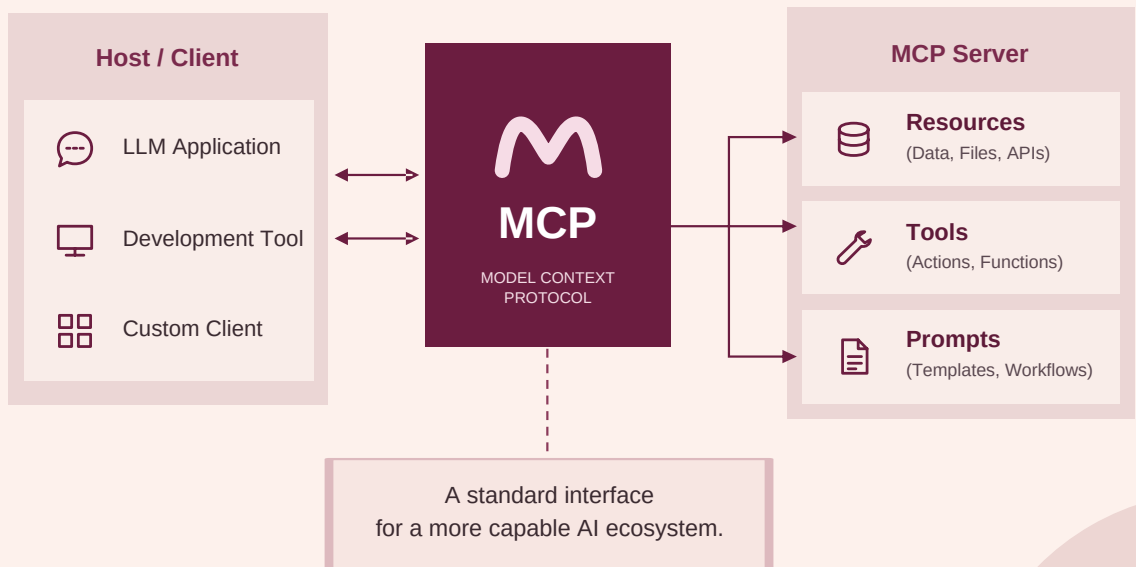


EXTEND

CHAPTER 04

The MCP Lifecycle

Initialization, operation, and shutdown: the complete request/response choreography behind every MCP session.



Premium Revision Notes

CONCEPTS

Clear Explanations

IMPLEMENTATION

Practical Code

DIAGRAMS

Visual Learning

INTERVIEW PREP

Key Questions

BUILD
CONNECT
EXTEND



SMARTER AI.
BIGGER POSSIBILITIES.

The MCP Lifecycle

Every MCP connection, regardless of transport or the tools it exposes, follows the same disciplined choreography from the moment a host launches a server to the moment the connection closes. This choreography is the **MCP lifecycle**: a fixed sequence of steps that governs how a host and a server establish, use, and end a session. Understanding it is what makes the difference between reading MCP traffic as noise and reading it as a predictable protocol.

1. What the Lifecycle Describes

The MCP lifecycle describes the complete sequence of steps that govern how a host and a server establish, use, and end a connection during a *session*. A session is one continuous connection between a client and a server — for example, the entire duration for which a desktop AI application keeps a filesystem server running, from the moment the application starts until it is closed.

Everything that happens inside that duration — the handshake, every tool call, every notification — is governed by the lifecycle. It breaks down into three stages, always in the same order.



Figure 4.1 — The three stages of the MCP lifecycle: initialization, operation, and shutdown.

IMPORTANT

Initialization must always be the very first interaction between a client and a server. No tool call, resource read, or prompt request may occur before it completes.

🔄 Quick Revision

- A **session** is one continuous connection between a client and a server for the lifetime of that connection.
- The lifecycle has exactly three stages: **initialization**, **operation**, **shutdown**.
- Initialization must always happen first and complete successfully before any other communication.

2. Initialization Phase

Initialization is the handshake: the client and server meet for the first time, agree on a protocol version, and declare what each of them is capable of. It always happens in exactly three steps.

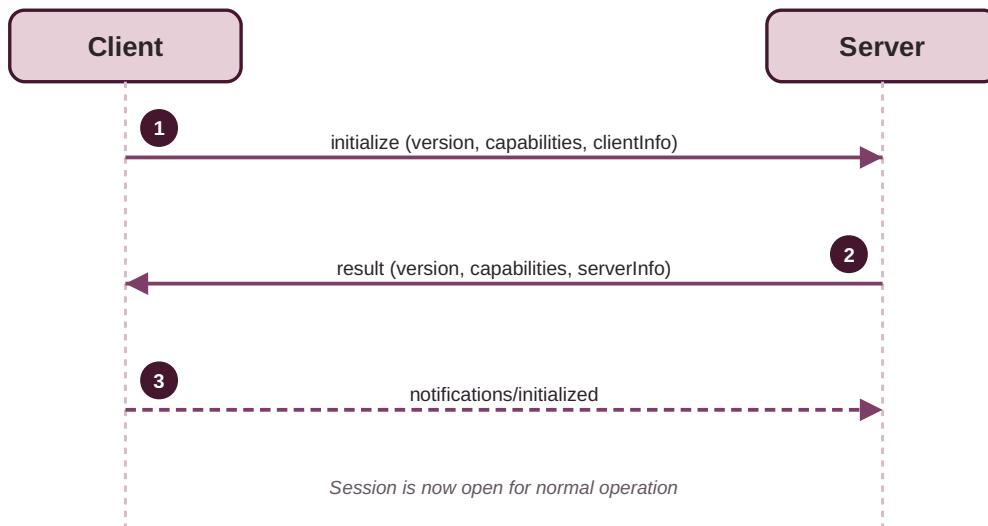


Figure 4.2 — The three-step initialization handshake between client and server.

Step 1 — Client Sends `initialize`

The client opens the session by calling the `initialize` method and sending three pieces of information: the MCP protocol version it speaks, the capabilities it offers the server (such as `roots` and `sampling`), and implementation info identifying the client and its version.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "roots": { "listChanged": true },
      "sampling": {}
    },
    "clientInfo": { "name": "IDEPlugin", "version": "1.0.0" }
  }
}
```

Step 2 — Server Responds

The server replies with the same three categories of information from its own side: the protocol version it will operate on, the capabilities it exposes (such as `tools`, `resources`, and `prompts`), and its own implementation info.

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2025-03-26",
    "capabilities": {
      "tools": { "listChanged": true },
      "resources": { "listChanged": true, "subscribe": true }
    },
    "serverInfo": { "name": "FileSystemServer", "version": "2.5.1" },
    "instructions": "Server is ready to accept commands"
  }
}
```

Step 3 — Client Confirms

Once the client has a successful response, it sends an `initialized` notification confirming the connection is ready for normal use. Being a notification, it carries no `id` and expects no reply.

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

After this notification is sent, the client and server are connected for the remainder of the session, and the client may request any operation the server has advertised.

COMMON MISTAKE

Treating `initialize` as just another request. It is special: neither side may send any request other than a ping before the handshake completes, and the server must not process anything but pings or log statements before receiving `notifications/initialized`.

🔄 Quick Revision

- Step 1: client calls `initialize` with its protocol version, capabilities, and implementation info.
- Step 2: server responds with its own protocol version, capabilities, and implementation info.
- Step 3: client sends the `notifications/initialized` notification — no reply expected.
- Before Step 3, the client may send nothing but a ping; the server may serve nothing but pings and log statements.

3. Version Negotiation

Because MCP continues to evolve, multiple protocol versions can exist at once. Version negotiation is how the client and server agree on which one to use for the session.

The client checks the version the server returned in its `initialize` response against a locally maintained list of versions it supports:

```
# client-side check
SUPPORTED_PROTOCOL_VERSIONS = ["2025-03-26", "2024-11-05"]
```

Outcome	Client Action
Server's version is in the client's supported list	Client sends <code>notifications/initialized</code> ; session proceeds normally
Server's version is not supported	Client disconnects; no further communication occurs

🔄 Quick Revision

- Version negotiation happens during initialization, right after the server's response arrives.
- The client compares the server's declared `protocolVersion` against its own supported-versions list.
- A mismatch causes the client to disconnect before any further messages are exchanged.

4. Capability Negotiation

Capability negotiation is the other half of initialization: it establishes which protocol features will actually be available during the session, so each side knows exactly what it can ask of the other.

Client and Server Capabilities

Each side advertises a distinct set of capabilities during initialization — the client offers three, the server offers four.

Side	Capability	Purpose
Client	roots	Gives the server access to a base directory on the host's filesystem (for example, the current project folder).
Client	sampling	Lets the server ask the client's own AI model to do work on its behalf, since MCP is a bidirectional protocol.
Client	elicitation	Lets the server request more information from the client when a prior request was incomplete (for example, a missing API key).
Server	tools	Callable functions the client can invoke to perform actions.
Server	resources	Static or semi-static content the client can fetch, such as files or documents.
Server	prompts	Reusable templates that guide how the server should be used.
Server	logging	Lets the server send status/log statements to the client, useful for long-running tasks.

Sub-Capabilities

Two sub-capabilities refine how **tools**, **resources**, and **prompts** behave during the session:

listChanged — server notifies the client when its tool/resource list changes

subscribe — client can watch a specific resource for updates

REAL WORLD INSIGHT

If a server advertises `resources: { "subscribe": true }`, a client that has subscribed to a file resource (say, a repository's README) will receive a change notification the moment that file is modified — without polling.

🔄 Quick Revision

- Client capabilities: **roots**, **sampling**, **elicitation**.
- Server capabilities: **tools**, **resources**, **prompts**, **logging**.
- Sub-capabilities **listChanged** and **subscribe** govern how change notifications are delivered.
- Capability negotiation sets expectations: each side may only use what the other side has actually advertised.

5. Operation Phase

Once initialization completes, the session enters the operation phase — the stage where the actual work happens. Two rules govern everything here: the negotiated protocol version must be respected, and only capabilities that were successfully negotiated may be used.

The operation phase itself splits into two parts.

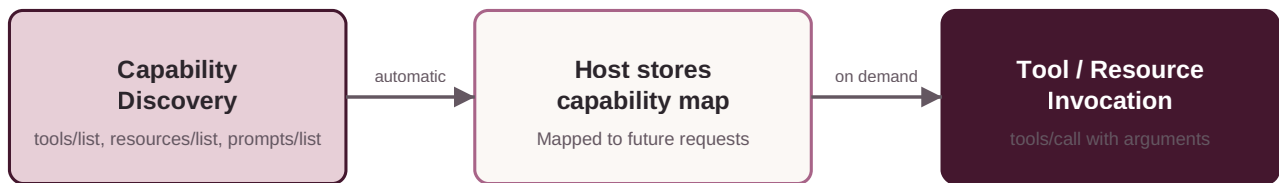


Figure 4.3 — Operation phase: capability discovery happens automatically, then tools and resources are invoked on demand.

Part 1 — Capability Discovery

Immediately after initialization, the client automatically asks the server exactly which tools, resources, and prompts it offers — the earlier handshake only confirmed which *categories* of capability exist, not their specifics.

```
{ "jsonrpc": "2.0", "id": 2, "method": "tools/list" }
```

```
{
  "jsonrpc": "2.0", "id": 2,
  "result": {
    "tools": [
      { "name": "listRepos", "description": "List user/org repositories" },
      { "name": "getFile", "description": "Read a file from a repo (path@ref)" },
      { "name": "searchCode", "description": "Search code across repos" },
      { "name": "createIssue", "description": "Open a GitHub issue" },
      { "name": "listPRs", "description": "List pull requests for a repo" }
    ]
  }
}
```

The host stores this list and later maps each incoming user request to the tool best suited to satisfy it.

Part 2 — Tool Calling

Once the exact catalog is known, the client can invoke any specific tool by name via `tools/call`, supplying whatever arguments that tool requires.

```
{
  "jsonrpc": "2.0", "id": 3,
  "method": "tools/call",
  "params": {
    "name": "getFile",
    "arguments": { "owner": "org", "repo": "mcp-examples", "path": "README.md", "ref":
"main" }
  }
}
```

```
{
  "jsonrpc": "2.0", "id": 3,
  "result": { "content": "# MCP Examples..." }
}
```

BEST PRACTICE

Treat the result of capability discovery as a live catalog, not a one-time snapshot — if a server advertises `listChanged`, re-fetch the list when notified rather than assuming it stays fixed for the session.

🔄 Quick Revision

- Operation phase = capability discovery, automatically triggered right after initialization, followed by on-demand invocation.
- Discovery uses `tools/list`, `resources/list`, and `prompts/list`.
- Invocation uses `tools/call` with the tool name and its arguments.
- A request for a capability that was never negotiated returns a *Method not found* error.

6. Shutdown Phase

Shutdown terminates the session. Unlike initialization and operation, shutdown involves **no JSON-RPC messages at all** — it is handled entirely by the transport layer, and the exact mechanism differs by transport.

Shutdown over STDIO (Local Servers)

For a locally running server, the client controls the server's process directly and shuts it down through an escalating sequence.



Figure 4.4 — Client-initiated STDIO shutdown: an escalating sequence from a polite close to a forced kill.

The client first closes the server's input stream and waits for it to exit gracefully. If the server does not exit, the client asks the operating system to send **SIGTERM** (signal terminate) — a polite request to shut down. If the server still does not exit, the client escalates to **SIGKILL** (signal kill), which forces termination.

IMPORTANT

Server-initiated shutdown over STDIO is rare. In that case the server simply closes its output stream and exits on its own; the client does not need to signal anything.

Shutdown over HTTP (Remote Servers)

For a remote server, shutdown is simpler: whichever side initiates it closes the underlying HTTP connection. If the client initiates (the common case), it terminates its own POST connection to the server. If the server initiates — typically due to an overload or an internal failure — it drops the connection from its side.

BEST PRACTICE

A client should always be prepared to handle a dropped HTTP connection gracefully — close any in-flight task cleanly and attempt to reconnect, since a server-initiated drop usually signals something went wrong mid-process.

🔄 Quick Revision

- Shutdown carries no JSON-RPC messages — it is entirely a transport-layer concern.
- STDIO: close stdin → wait → SIGTERM → wait → SIGKILL, if the server does not exit on its own.
- HTTP: whichever side initiates closes the connection directly; no escalation sequence exists.
- Clients should handle an unexpected server-side connection drop gracefully rather than assuming the server misbehaved.

7. Special Cases

Beyond the standard init → operate → shutdown flow, five recurring situations appear across real MCP sessions.

7.1 — Pings

A **ping** is a lightweight request/response method built into MCP to check whether the other side (host or server) is still alive and the connection is still responsive. Either side can send one, and the receiver must reply immediately with an empty result.

```
// request
{ "jsonrpc": "2.0", "id": 42, "method": "ping" }

// response
{ "jsonrpc": "2.0", "id": 42, "result": {} }
```

Pings are useful in two situations: confirming the other side is up before initialization completes, and keeping a connection alive during a long-running task so that an idle firewall, proxy, or OS does not silently drop it.

7.2 — Error Handling

MCP inherits its error handling directly from JSON-RPC 2.0's standard error object, so the same structure and codes recur across implementations.

```
{
  "jsonrpc": "2.0", "id": 5,
  "error": {
    "code": -32601,
    "message": "Method not found",
    "data": { "extra": "optional info" }
  }
}
```

Field	Meaning
code	Integer code categorizing the error
message	Short human-readable explanation
data	Optional structured detail useful for debugging

Common scenarios that produce an error include a protocol version mismatch during initialization, calling a method the other side never advertised, sending invalid arguments to a tool, an internal server failure, a request timing out, and malformed JSON-RPC syntax.

Code	Name	Meaning	Example
-32600	Invalid Request	The method being called does not exist	Hitting an endpoint the server never defined
-32601	Method not found	Called a method that doesn't exist or wasn't advertised	Host calls <code>prompts/list</code> but the server never advertised <code>prompts</code>
-32602	Invalid params	Request sent with wrong or missing parameters	Tool expects <code>{"path": "..."} </code> , client sends <code>{"file": "..."} </code>
-32700	Parse error	Request body is not valid JSON	Malformed JSON-RPC request
-32000 and above	Server-defined errors	Custom errors defined by the server (implementation-specific)	Authentication failure, rate limit exceeded, quota errors, internal issues

7.3 — Timeout

A **timeout** ensures a request does not hang forever. The client's SDK lets it set a per-request threshold; if the server does not respond within that window, the client stops waiting.

Timeouts protect against three things: an unresponsive or overloaded server, resources (memory, CPU) being held indefinitely for a response that may never arrive, and a user left with no feedback at all.

When a timeout triggers, the client sends a cancellation notification for the original request:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/cancelled",
  "params": { "requestId": 7, "reason": "Timeout exceeded (30s)" }
}
```

The server then stops processing that request and sends no further response for it.

7.4 — Progress Notification

For a long-running task, the client can attach a **progress token** to its request metadata. The server then periodically reports progress against that token, so the user gets real-time feedback instead of silence.

```
// request with a progress token attached
{
  "jsonrpc": "2.0", "id": 7,
  "method": "tools/call",
  "params": {
    "name": "searchCode", "arguments": { "query": "MCP lifecycle" },
    "_meta": { "progressToken": "tok-7" }
  }
}

// periodic progress notification
{
  "jsonrpc": "2.0",
  "method": "notifications/progress",
  "params": {
    "progressToken": "tok-7", "progress": 60, "total": 100,
    "message": "Searching 600 of 1000 files"
  }
}
```

INTERVIEW TIP

Be ready to explain the difference between a *ping* (checks liveness, no payload), a *progress notification* (reports percentage completion of an active task), and a *cancellation notification* (tells the other side to stop processing a specific request).

🔄 Quick Revision

- **Ping**: lightweight liveness check, either side can send it, expects an empty result.
- **Errors** follow the JSON-RPC 2.0 error object: **code**, **message**, optional **data**.
- Standard codes: -32700 parse error, -32600 invalid request, -32601 method not found, -32602 invalid params; -32000 and above are server-defined.
- **Timeout** triggers a **notifications/cancelled** message referencing the original request ID.
- **Progress notifications** report percentage completion against a client-supplied progress token.

8. Key Takeaways

- The MCP lifecycle has exactly three stages — initialization, operation, shutdown — always in that order.
- Initialization is a strict three-step handshake: `initialize` request, `server result`, `initialized` notification.
- Version negotiation and capability negotiation both happen inside the initialization phase.
- Operation splits into automatic capability discovery followed by on-demand tool/resource invocation.
- Shutdown involves no JSON-RPC messages — it is entirely a transport-layer event, with STDIO and HTTP handling it differently.
- Pings, structured errors, timeouts, cancellation, and progress notifications are the recurring special cases layered on top of the core flow.

9. Revision Sheet

A one-glance recap of every mechanism covered in this chapter.

Session

One continuous connection between a client and a server for its full duration.

Initialization

`initialize` → server result → `notifications/initialized`.

Version Negotiation

Client checks server's declared version against its own supported list; mismatch = disconnect.

Client Capabilities

roots, sampling, elicitation.

Server Capabilities

tools, resources, prompts, logging.

Operation Phase

Discovery (`*/list`) then invocation (`tools/call`).

Shutdown

Transport-level only — STDIO escalates `stdin close` → `SIGTERM` → `SIGKILL`; HTTP simply closes the connection.

Error Object

`code`, `message`, optional `data` — inherited from JSON-RPC 2.0.

10. Interview Questions

- Q1.** What are the three stages of the MCP lifecycle, and why must they occur in that order?
- Q2.** Walk through the three steps of the initialization handshake and explain what each message contains.
- Q3.** What happens if the client and server report different protocol versions during initialization?
- Q4.** Name the three client capabilities and the four server capabilities, and give one use case for each.
- Q5.** What is the difference between the `listChanged` and `subscribe` sub-capabilities?
- Q6.** Why does the shutdown phase involve no JSON-RPC messages?
- Q7.** Describe the escalation sequence a client follows to shut down a local STDIO server.
- Q8.** What is the purpose of a ping, and how does it differ from a progress notification?
- Q9.** What are the three fields of a standard MCP/JSON-RPC error object?
- Q10.** What happens on both the client and server side when a request times out?

11. Further Reading

- Model Context Protocol specification — Lifecycle section (protocolVersion negotiation, capability objects)
- JSON-RPC 2.0 specification — request, response, notification, and error object formats
- POSIX signal handling — SIGTERM vs. SIGKILL semantics for process termination